

Lab #4 EGRE246

For this lab you will update and modify your lab from last week. You are going to modify your program to expand your array of widgets “on the fly” dynamically at runtime as in the example on page 46 of the class notes; I have included this program in this handout (see reverse).

Perform the following tasks:

1. Comment out the routine `initDB` and id `MAX_DB_SIZE` and `count` in last week’s program - they will no longer be needed.
2. Define your database array `db` as a pointer (initially set to `NULL`). You will not need to modify function `newItem`.
3. Like in the example program on the back, define a preprocessor constant `EXPANSION_SIZE` with a value of 2.
4. Now in `main` instead of calling `initDB` (which no longer exists) to create your database you should `malloc` a new array in `main` with with `EXPANSION_SIZE` size and assign it to `db`. Model your code after the code on the reverse from the notes.
5. You will need to come up with a new way to keep track of the number of items in your database and the size of your array; for simplicity sake I suggest you just define global variables to keep track of these two values. Examine the variables `arraySize` and `listSize` in the program on the reverse; what are they used for?
6. After defining these 2 new global variables, modify your routine `printDB` to use them. Test your code using an arbitrary large number for the maximum size of the database; you can use the program `lab3Fall123GENERATOR.c` on Canvas to generate random data sets of any size (just redirect the data into a file on the command line and then use redirection in your program to input from this file, making sure you add the total number of items to the beginning of this file).
7. If everything works – FINISHED!

```

#include <stdio.h>
#include <stdlib.h>

#define EXPANSION_SIZE 10
#define DEBUG 1

int arraySize = 0, listSize = 0;

int *insert(int *list, int item) {
    int *l = list;
    if (listSize >= arraySize) {
        if (DEBUG) printf("DEBUG: expanding array by %d to %d\n",
                           EXPANSION_SIZE, arraySize+EXPANSION_SIZE);
        l = (int *) malloc((arraySize=arraySize+EXPANSION_SIZE)*sizeof(int));
        if (list != NULL) {
            for(int i=0; i < listSize; i++) l[i] = list[i];
            free(list);
        }
    }
    l[listSize++] = item;
    return l;
}

int main(void){
    int *list=NULL, i=0;
    for (i=1; i <= 21; i++) list = insert(list, i*2);
    for(i = 0; i < listSize; i++) printf("%d ", list[i]);
    printf("\n");
}

```